Muddiest Points I

Frage: *Mir ist noch nicht ganz klar, was alles ins Codebook soll und wie dieses aufgebaut sein soll. Mir ist noch etwas unklar, wie man beim Ausfüllen des Codebooks konkret vorgeht, wo man die nötigen Angaben pro Variable jeweils findet und wie man diese dann im Codebook einträgt. Könnten wir allenfalls nochmals ein paar Variablen gemeinsam durchgehen?*

Antwort: In EH5 werden wir gemeinsam anhand der Variable *strategies* Schritt für Schritt durchgehen, wie ein vollständiger Codebook-Eintrag erstellt wird. Ausserdem werdet ihr in Hands-on 3 angeleitet, eine weitere Variable selbstständig zu untersuchen und zu dokumentieren. Dabei erklären wir auch die einzelnen Felder des Codebooks und die Verwendung der Response Labels.

Grundsätzlich findet ihr die benötigten Informationen in den bereitgestellten Studienunterlagen (ZIP-Ordner). Besonders hilfreich sind dabei folgende Quellen:

- **Datensatz dat_full:** Hier könnt ihr in R die vorhandenen Variablen, ihre Datentypen und die tatsächlich vorkommenden Werte untersuchen, bspw. mit class(), unique(), table() oder range().
- **Informationen_zu_Daten.xlsx:** Diese Datei enthält wichtige Informationen zur Bedeutung und Kodierung der einzelnen Variablen sowie zu den verwendeten Antwortkategorien.
- **Originalpaper von Grinschgl et al.:** Insbesondere im Methodenteil findet ihr Informationen darüber, wie die Variablen erhoben wurden, welche Aufgaben und Fragebögen verwendet wurden und was die einzelnen Messwerte bedeuten.
- **Codebook-Guideline:** Hier wird erklärt, welche Informationen in den einzelnen Feldern des Codebooks eingetragen werden sollen und was dabei zu beachten ist.

Am besten geht ihr Variable für Variable vor: Zunächst untersucht ihr die Variable in R, anschliessend sucht ihr die entsprechenden Informationen in den Studienunterlagen und übertragt diese in die passende Zeile des Codebooks. Bei kategorialen Variablen und Fragebogenitems müssen ggf. zusätzlich die Antwortkategorien im Tabellenblatt response_labels dokumentiert werden.

Wichtig: Für die erste Codebook-Abgabe dokumentiert ihr den aktuellen Stand von dat_full nach EH4. Die Transformationen, die wir in den folgenden Einheiten durchführen, werden anschliessend im Codebook nachgeführt.

Das Ziel ist, dass eine andere Person mithilfe eures Codebooks den Datensatz und seine Variablen verstehen und korrekt interpretieren kann, ohne zusätzliche Erklärungen von euch zu benötigen.

Frage: *Die Unterschiede zwischen den verschiedenen Merge-Funktionen.*

Antwort: Mit den Join-Funktionen aus dem Paket dplyr können wir zwei Datensätze anhand einer oder mehrerer gemeinsamer Schlüsselvariablen (z.B. einer Personen-ID code) zusammenführen. **Der wichtigste Unterschied besteht darin, welche Zeilen aus den beiden Datensätzen übernommen werden.**

Nehmen wir an, wir haben zwei Datensätze mit folgenden Personen-IDs:

- Datensatz A: 1, 2, 3
- Datensatz B: 2, 3, 4

Die verschiedenen Join-Funktionen unterscheiden sich folgendermassen:

Funktion	**Welche Zeilen werden übernommen?**	**IDs im Ergebnis**
full_join(A, B, by = "code")	Alle Zeilen aus beiden Datensätzen.	1, 2, 3, 4
left_join(A, B, by = "code")	Alle Zeilen aus A. Passende Informationen aus B werden ergänzt.	1, 2, 3
right_join(A, B, by = "code")	Alle Zeilen aus B. Passende Informationen aus A werden ergänzt.	2, 3, 4
inner_join(A, B, by = "code")	Nur Zeilen, deren Schlüssel in beiden Datensätzen vorkommen.	2, 3
semi_join(A, B, by = "code")	Nur Zeilen aus A, für die ein passender Schlüssel in B existiert. Es werden keine Spalten aus B ergänzt.	2, 3
anti_join(A, B, by = "code")	Nur Zeilen aus A, für die kein passender Schlüssel in B existiert. Es werden keine Spalten aus B ergänzt.	1

Bei den ersten vier Funktionen werden Spalten aus beiden Datensätzen kombiniert. Gibt es zu einer übernommenen Person keine passende Information im anderen Datensatz, entstehen für diese Spalte fehlende Werte (NA).

semi_join() und anti_join() funktionieren hingegen als Filter: Sie überprüfen, welche Personen in beiden Datensätzen bzw. nur im ersten Datensatz vorkommen. Sie ergänzen keine neuen Spalten.

Welche Funktion sollte ich verwenden?

Das hängt davon ab, was du erreichen möchtest:

- Möchtest du möglichst alle Personen aus beiden Datensätzen behalten? → full_join()

- Möchtest du einen Hauptdatensatz behalten und zusätzliche Informationen ergänzen? → left_join() oder right_join()
- Möchtest du nur Personen behalten, die in beiden Datensätzen vorkommen? → inner_join()
- Möchtest du prüfen, welche Personen im anderen Datensatz vorhanden sind oder fehlen? → semi_join() bzw. anti_join()

Wichtig: Die Schlüsselvariable sollte vor dem Zusammenführen kontrolliert werden. Wenn eine Personen-ID mehrfach vorkommt, können bei den ersten vier Join-Funktionen unerwartet zusätzliche Zeilen entstehen.

Bezug zu unserem Forschungsprojekt: In EH4 haben wir full_join() verwendet, weil wir die Informationen aus den sieben Rohdatensätzen zusammenführen wollten, ohne Personen zu verlieren, die möglicherweise nur in einzelnen Dateien vorkommen. Anschliessend haben wir überprüft, ob der Merge korrekt funktioniert hat.

Frage: *Bei der Hands-on-Übung in Block 2 sind für mich noch einige Fragen zum Arbeiten mit Quarto offen:*

- *Wie richte ich den YAML-Header korrekt ein, insbesondere im Hinblick auf die Einrückungen?*
- *Welche Vorteile haben der Visual Editor und der Source Editor, und wann eignet sich welcher?*
- *Wie lässt sich das Rendern sinnvoll in den Arbeitsablauf integrieren? Ich bin mir noch nicht ganz sicher, wann ich während der Bearbeitung eines Datensatzes rendern sollte und welchen Zweck das Rendern über die Prüfung der Reproduzierbarkeit hinaus erfüllt.*

Antwort:

1. YAML-Header und Einrückungen

Der YAML-Header steht ganz am Anfang eines Quarto-Dokuments (.qmd) zwischen zwei Zeilen mit jeweils drei Bindestrichen (---). Er enthält grundlegende Einstellungen zum Dokument, bspw. den Titel, das Ausgabeformat oder die Darstellung des R-Codes.

Ein einfaches Beispiel:

```
---
title: "Datenaufbereitung"
format:
  html:
    toc: true
execute:
  echo: true
---
```

Die Einrückungen zeigen, welche Einstellungen zusammengehören:

- title, format und execute stehen auf derselben Ebene und sind deshalb nicht eingerückt.
- html ist eine Untereinstellung von format und wird deshalb um zwei Leerzeichen eingerückt.
- toc gehört wiederum zu html und wird um zwei weitere Leerzeichen eingerückt.
- echo gehört zu execute und wird entsprechend eingerückt.

Wichtig ist, konsequent Leerzeichen statt Tabulatoren zu verwenden. Falsche Einrückungen können dazu führen, dass Quarto die Einstellungen nicht korrekt interpretiert oder beim Rendern eine Fehlermeldung ausgibt.

Für unser Seminar sind die YAML-Header in processing.qmd und analysis.qmd bereits vorbereitet. Ihr müsst diese also nicht selbstständig von Grund auf erstellen, sondern bloss verstehen und an entsprechenden Stellen anpassen.

2. Visual Editor und Source Editor

RStudio bietet zwei Möglichkeiten, Quarto-Dokumente zu bearbeiten. Beide arbeiten mit derselben .qmd-Datei und ihr könnt jederzeit zwischen ihnen wechseln. Beide Editoren bieten unterschiedliche Ansichten und Bearbeitungsmöglichkeiten für dieselbe .qmd-Datei.

Visual Editor	**Source Editor**
Zeigt Text und Formatierungen ähnlich wie in einem Textverarbeitungsprogramm an.	Zeigt den zugrunde liegenden Markdown- und Quarto-Quelltext.
Praktisch zum Schreiben, Formatieren und Strukturieren von Texten und Tabellen.	Praktisch zum Bearbeiten von R-Code, YAML-Einstellungen und komplexeren Quarto-Elementen.
Viele Formatierungen lassen sich über Schaltflächen vornehmen.	Ermöglicht die direkte Kontrolle über die genaue Schreibweise und Struktur.

3. Wann und warum sollte ich rendern?

Beim Rendern verarbeitet Quarto das gesamte Dokument und erstellt daraus bspw. eine HTML-Datei. Dabei werden die R-Code-Chunks in ihrer Reihenfolge ausgeführt und ihre Ergebnisse, etwa Tabellen, Kennwerte oder Abbildungen, in das fertige Dokument integriert.

Das Rendern erfüllt somit mehrere Zwecke:

- **Reproduzierbarkeit überprüfen:** Funktioniert das Skript vollständig von Anfang bis Ende, ohne dass zuvor manuell Objekte im R-Environment erstellt werden müssen?
- **Fehler erkennen:** Fehlende Objekte, falsche Pfade oder Probleme mit der Reihenfolge des Codes werden sichtbar.
- **Ergebnisse kontrollieren:** Ihr könnt überprüfen, ob Tabellen, Abbildungen, Berechnungen und Erklärungen korrekt und verständlich dargestellt werden.

- **Ergebnisse dokumentieren und teilen:** Aus dem Quellcode entsteht ein lesbares Forschungsdokument, das Verarbeitungsschritte und Ergebnisse gemeinsam festhält.

Wann sollte ich rendern?

Während ihr an einer einzelnen Aufgabe arbeitet, müsst ihr nicht nach jedem neuen Befehl das gesamte Dokument rendern. Ihr könnt zunächst einzelne Codezeilen oder Chunks ausführen, die Ergebnisse überprüfen und euren Code schrittweise entwickeln.

Ich empfehle aber, nach grösseren abgeschlossenen Arbeitsschritten das vollständige Dokument zu rendern, bspw. nachdem ihr die sieben Rohdatensätze zusammengeführt oder einen neuen Abschnitt der Datenaufbereitung fertiggestellt habt.

Spätestens vor der Abgabe sollten beide Quarto-Dokumente vollständig und fehlerfrei rendern.

Wichtig: Das Rendern speichert nicht automatisch jedes in R erzeugte Datenobjekt als separate Datei. Wenn bspw. dat_full.csv gespeichert werden soll, muss im Processing-Skript ein entsprechender Speicherbefehl wie write_csv() enthalten sein.

Frage: *Mehr zu YAML-Header und HTML-Dateien.*

Antwort:

1. Welche weiteren Funktionen bietet der YAML-Header?

Neben den bereits besprochenen Grundeinstellungen könnt ihr im YAML-Header zahlreiche weitere Eigenschaften eures Quarto-Dokuments festlegen. Besonders nützlich sind bspw.:

Einstellung	**Funktion**
author	Gibt die Autorin oder den Autor des Dokuments an.
date	Fügt ein Datum hinzu.
theme	Bestimmt das visuelle Erscheinungsbild der HTML-Datei.
number-sections	Nummeriert die Überschriften automatisch.
code-fold	Ermöglicht es, R-Code im HTML-Dokument ein- und auszublenden.
warning / message	Steuert, ob Warnungen bzw. Mitteilungen aus R im fertigen Dokument angezeigt werden.

Bspw. könnt ihr mit code-fold: true dafür sorgen, dass euer R-Code im HTML-Dokument zunächst eingeklappt ist und bei Bedarf angezeigt werden kann.

Wichtig: Der YAML-Header steuert v.a. die Darstellung und Ausführung des Dokuments. Er ersetzt nicht die eigentlichen R-Befehle zur Datenaufbereitung oder -analyse. Ausserdem sollten relevante Warnungen nicht einfach ausgeblendet werden, da sie auf mögliche Probleme im Code hinweisen können.

2. Was ist eine HTML-Datei und welche Vorteile bietet sie?

HTML (HyperText Markup Language) ist ein Format zur Darstellung von Inhalten in einem Webbrowser.

Wenn ihr euer Quarto-Dokument als HTML rendert, entsteht ein übersichtlich formatiertes Dokument, das verschiedene Inhalte miteinander verbinden kann:

- **Texte und Überschriften:** Eure Erläuterungen und die Struktur des Forschungsprojekts.
- **R-Code:** Die verwendeten Befehle, die bei Bedarf ein- und ausgeblendet werden können.
- **Ergebnisse:** Berechnete Kennwerte, Tabellen und Abbildungen.
- **Navigation:** Bspw. ein automatisch erstelltes Inhaltsverzeichnis, über das einzelne Abschnitte direkt aufgerufen werden können.

Ein wesentlicher Vorteil ist, dass andere Personen die fertige HTML-Datei normalerweise direkt im Browser betrachten können, ohne selbst R oder RStudio installiert zu haben. So können sie eure dokumentierten Verarbeitungsschritte, Berechnungen und Ergebnisse nachvollziehen.

3. Was ist der Unterschied zwischen .qmd und .html?

Die beiden Formate erfüllen unterschiedliche Aufgaben:

.qmd-Quelldokument	**.html-Ergebnisdokument**
Enthält eure bearbeiteten Texte, R-Code-Chunks und Quarto-Einstellungen.	Zeigt das fertig formatierte Dokument mit den beim Rendern erzeugten Ergebnissen.
Hier entwickelt und verändert ihr eure Datenaufbereitung und -analyse.	Hier können andere Personen eure Arbeit bequem lesen und überprüfen.

Für unser Seminar bedeutet das: Ihr arbeitet grundsätzlich in processing.qmd und analysis.qmd. Die daraus erzeugten HTML-Dateien dienen der verständlichen Darstellung und Dokumentation eurer Arbeit.

Wenn ihr nachträglich etwas verändern möchtet, passt ihr das entsprechende .qmd-Dokument an und rendert es erneut. Die HTML-Datei sollte nicht manuell bearbeitet werden.

Für das Abschlussprojekt benötigt ihr deshalb sowohl die Quarto-Quelldateien als auch die daraus erzeugten HTML-Dateien. Zusammen ermöglichen sie, die Vorgehensweise nachzuvollziehen und die Ergebnisse verständlich darzustellen.

Frage: *Ordnerstruktur: Was genau sollen wir alles in dem Ordner ablegen, den wir am Schluss abgeben? Konkret: Sollen wir z.B. das erhaltene und das gegebene Peer-Feedback auch in diesem Ordner ablegen? Sollen wir die erste Hands-on-Übung auch in diesem Ordner speichern?*

Antwort: Für das Abschlussprojekt sollt ihr nur diejenigen Dateien abgeben, die unmittelbar zu eurem Forschungsprojekt mit den individuell simulierten Daten gehören. Die Übungen aus dem Semester sowie das gegebene und erhaltene Peer-Feedback müssen nicht in den Abgabeordner aufgenommen werden.

Für das Abschlussprojekt erhaltet ihr nochmals eine separate ZIP-Datei mit der vorgegebenen Projekt- und Ordnerstruktur sowie allen relevanten Unterlagen. Diese Struktur entspricht grundsätzlich derjenigen, die wir während des Semesters für die gemeinsame Replikation der Grinschgl-Studie verwenden.

In dieser Projektstruktur bearbeitet bzw. ergänzt ihr insbesondere folgende Bestandteile:

- **Datenanalyseplan:** Den bereits während des Semesters erstellten Datenanalyseplan überarbeitet ihr und passt ihn an die individuell simulierten Daten sowie die möglichen Besonderheiten und Fehlerquellen an. Die überarbeitete Version legt ihr im entsprechenden Ordner ab.
- **Codebook:** Auch das Codebook habt ihr während des Semesters bereits erstellt. Für die Abschlussarbeit aktualisiert ihr es so, dass es sämtliche Variablen des final aufbereiteten Wide-Datensatzes eurer simulierten Daten korrekt dokumentiert.
- **processing.qmd:** Hier dokumentiert ihr die vollständige Aufbereitung eurer simulierten Rohdaten. Das Skript soll die bereinigten Datensätze dat_full.csv und dat_long.csv erzeugen.
- **analysis.qmd:** Hier führt ihr mit den aufbereiteten Daten die geforderten statistischen Analysen durch und interpretiert deren Ergebnisse.
- **HTML-Dateien:** Beide Quarto-Dokumente müssen zusätzlich als HTML (Word oder PDF auch möglich) gerendert und abgegeben werden.

Die bereitgestellten Rohdatensätze und die erzeugten aufbereiteten Datensätze bleiben ebenfalls in den dafür vorgesehenen Ordnern. Nicht mehr benötigte Vorlagen hingegen können entfernt werden.

Wichtig: Ihr müsst die Ordnerstruktur nicht selbst neu erstellen. Verwendet die Struktur der ZIP-Datei und ergänzt bzw. aktualisiert die benötigten Dateien an den vorgesehenen Stellen.

Nicht in die Abschlussabgabe gehören die separaten Hands-on-Übungen, die R-Hausübungen aus dem Semester oder das gegebene oder erhaltene Peer-Feedback. Ihr

gebt am Ende ein vollständiges, nachvollziehbares und reproduzierbares Forschungsprojekt mit euren individuell simulierten Daten ab – keine Sammlung sämtlicher Unterlagen und Übungen aus dem Semester.